Tech Nova Solutions – Project

Table of Contents

1. Problem Statement

Modern software companies require a professional online presence that not only showcases their work but also provides interactive client engagement. Many existing solutions are static, lack automation, and do not integrate AI support for user interaction. Tech Nova Solutions addresses this by providing a dynamic portfolio platform with secure user management, an AI chatbot, and administrative oversight.

2. Project Overview

Tech Nova Solutions is a comprehensive full-stack web application developed to function as a corporate portfolio and client interaction platform. It represents a modern software house by showcasing company services, completed projects, and team expertise in a structured and professional manner.

The application goes beyond a static website by integrating multiple dynamic and interactive features. It implements secure user authentication, role-based access control, an AI-powered chatbot assistant, a contact management system, and a protected administrative dashboard for system monitoring and management.

The system is developed using the **Flask** framework and follows the **Model–View–Controller (MVC)** architectural pattern. This architecture ensures a clear separation between business logic, user interface, and data handling, making the system scalable, maintainable, and efficient.

It provides a **user-friendly interface** for visitors and integrates an **AI-powered chatbot** for customer engagement and support.

Key Features of the Platform:

- Portfolio and project showcase
- User contact form for inquiries
- AI-powered chatbot for automated support and FAQs
- Admin dashboard for monitoring user messages and chatbot interactions
- Secure authentication system

Role-based access control for administrators and users

3. Objectives

The primary objectives of this project are:

- To provide a **digital portfolio** platform for showcasing services and projects.
- To integrate **AI capabilities** for automated user support.
- To ensure **efficient data management** for messages, users, and chat logs.
- To implement **role-based authentication** to restrict access to admin functionalities.
- To develop a **scalable and secure solution** for potential future enhancements.
- Provide user registration and login functionalities.

4. Scope of the Project

- Public visitors can browse projects, services, and company information.
- Users can submit inquiries through a contact form.
- AI chatbot provides immediate responses to user questions.
- Administrators can monitor and manage messages and chatbot interactions.
- The system can be scaled to integrate additional features like project uploads, analytics dashboards, or advanced AI modules.

5. Non-Functional Requirements

- **Performance:** The system should respond to user queries and AI requests within 2–3 seconds.
- **Security:** Authentication, session management, and role-based access control must be enforced.
- **Scalability:** Database and server structure should support growth in users and messages.
- **Usability:** Intuitive UI for users of all technical levels.
- **Maintainability:** Code structured using MVC for easier future updates.

6. Technologies and Tools Used

Technology	Purpose
Python 3.10+	Backend development and server-side logic
Flask	Web framework for routing, templating, and session management
SQLite3	Lightweight relational database for data storage
HTML / CSS / Bootstrap / JavaScript	Frontend design, responsiveness, and interactivity
Google Gemini API	AI-based chatbot response generation
Jinja2 Templates	Dynamic rendering of HTML pages
Datetime Module	Handling timestamps for messages and chat logs
Functools Module	Creating authentication decorators and route protection

-

7. System Architecture

The application follows a **three-tier architecture**:

1. **Presentation Layer (Frontend)**
    - Handles user interface through HTML, CSS, Bootstrap, and JavaScript.
    - Includes pages for home, portfolio, contact, login, registration, and admin dashboard.
2. **Business Logic Layer (Backend)**
    - Manages routing, authentication, and form submissions using Flask.
    - Integrates AI responses via Google Gemini API.
    - Implements role-based access control.
3. **Data Layer (Database)**

- Uses SQLite to store messages, chat logs, and user information.
- Ensures data persistence for all user interactions.

8. MVC Architectural Implementation

- **Model:** Handles database operations (users, messages, chat logs) using SQLite.
- **View:** HTML templates rendered via Jinja2, displaying data to users.
- **Controller:** Flask routes manage logic, handle requests, and direct data between model and view.

9. Database Design (Schema)

9.1 Users Table

Column	Description
id	Primary key, auto-increment
first name	First name of user
last-named	Last name of user
email	Unique email used for login
password	User password (encrypted in production)
role	Role designation: user or admin

9.2 Messages Table

Column	Description
id	Primary key, auto-increment
name	Name of sender
email	Email of sender
subject	Message subject

message	Content of the message
date	Timestamp of submission

9.3 Chat Logs Table

Column	Description
id	Primary key, auto-increment
user query	Message sent by the user
Bot response	Response generated by AI chatbot
timestamp	Timestamp of the interaction

10. Functional Features

The **Tech Nova Solutions** platform provides structured functionalities for users, administrators, and AI-based customer support. These features ensure smooth interaction, secure management, and intelligent automation.

10.1 User Features

Portfolio Browsing

- Users can explore company services and completed projects.
- Projects are displayed in an organized and visually appealing format.
- Helps visitors understand the company's expertise and capabilities.

Contact Form Submission

- Users can submit inquiries or feedback through a contact form.
- Messages are securely stored in the database.
- Enables direct communication between clients and the company.

AI Chatbot

- Users can interact with an AI-powered chatbot for instant support.
- Provides automated replies for common questions.
- Enhances customer engagement and user experience.

Registration & Login

- Users can create secure accounts.
- Authentication system verifies login credentials.
- Logged-in users can access personalized features.

10.2 Admin Features

Dashboard Access

- Admins can access a secure administrative dashboard.
- View all submitted contact messages.
- Monitor chatbot interaction logs.

Role-Based Access Control

- Only users with the **admin** role can access dashboard functionalities.
- Prevents unauthorized access to sensitive data.
- Ensures system security and proper privilege management.

Monitoring & Analytics

- Admins can track user inquiries and chatbot usage.
- Helps analyze support trends and common queries.
- Improves decision-making and service enhancement.

10.3 AI Chatbot Features

Rule-Based Responses

- Provides automated replies for common queries such as services, contact details, and working hours.
- Ensures fast and consistent responses.

AI-Powered Responses

- Complex or advanced queries are handled using the **Google Gemini API**.
- Generates contextual and intelligent responses.
- Improves the accuracy and quality of customer support.

Interaction Logging

- All chatbot conversations are stored in the database.
- Enables review, monitoring, and performance analysis.
- Supports future improvements in chatbot behavior.

11. Feature Walkthrough

11.1 Public Interface

1. Home: Hero section with animated text and social links.
2. About: Grid layout displaying company information.
3. Services: Card-based layout showcasing 8 different services (Front-end, UI/UX, etc.) with specific icons.
4. Projects: Portfolio grid displaying recent work with links to GitHub repositories and live demos.
5. Contact: A functional form validated on the client side and submitted to the backend.

11.2 AI Chatbot Widget

A floating chat bubble fixed at the bottom right. It opens a chat window allowing users to ask questions. The bot responds using either pre-defined answers or AI-generated text, simulating a real support agent.

11.3 Admin Dashboard

Accessibly only by logging in as an Admin.

- Credentials: **admin@technova.com** / **admin123**.
- Functionality: Displays two data tables:
    1. Contact Messages: Name, Email, and Message content from visitors.
    2. Chatbot History: Full transcript of all chatbot conversations for analysis.

12. Frontend Implementation Details

12.1 Template Inheritance (**base.html**)

The project utilizes Jinja2's template inheritance to avoid code duplication.

- Blocks: Defines blocks (**title**, **content**, **extra_head**, **extra_scripts**) that child templates override.
- Global Elements: Contains the Header, Navigation Bar, Flash Message container, and Footer.
- Responsive Navigation: Includes the HTML structure for the mobile hamburger menu and overlay.

12.2 Styling Strategy (style.css)

The CSS is organized into sections using comments for clarity:

- CSS Variables: Defined in **:root** for consistent theming (colors like **--bg-primary**, **--accent**).
- Responsive Design: Uses Media Queries (**@media**) to adjust layouts for Desktops, Tablets, and Mobile devices. For example, the navigation bar transforms into a slide-out sidebar on screens smaller than 768px.
- Animations: Keyframe animations (**slideTop**, **floatImage**) provide a modern, dynamic feel to the homepage elements.

12.3 Client-Side Logic (**script.js**)

- Typed.js Integration: Initializes the typing effect on the homepage hero section ("We Build Web Applications...").
- Sidebar Control: Manages the toggle logic for the mobile navigation menu, handling classes **active** and updating ARIA attributes for accessibility.
- Chatbot Interaction:
  - **sendMessage()**: Asynchronously sends user input to the **/chatbot** endpoint using **fetch**.
  - **addMessage()**: Dynamically creates DOM elements to display the chat history without reloading the page.

13. Backend Implementation (API Endpoints)

Route	Method	Description
/	GET	Homepage
/portfolio	GET	Portfolio page
/login	GET/POST	Login page
/register	GET/POST	Registration page
/logout	GET	Logout
/submit_contact	POST	Contact form submission
/chatbot	POST	Chatbot interaction
/admin/dashboard	GET	Admin dashboard (admin only)

14. Security Measures

1. Session Management: Uses Flask's secure session object to track user login state. The secret key signs the session cookie to prevent tampering.
2. Route Protection: Sensitive routes (Admin Dashboard) are wrapped in the **@login_required** decorator, ensuring only authorized personnel can access backend data.
3. SQL Injection Prevention: The code uses parameterized queries (e.g., **?** placeholders) when interacting with the SQLite database. This prevents malicious SQL code from being injected via input forms.
4. Passwords should be stored in **hashed form** in production.
5. API keys should be stored in **environment variables** for security.
6. All form inputs should be validated and sanitized to prevent SQL injection and other attacks.
7. Role-based access ensures **restricted admin functionality**.

15.Installation and Setup Guide

To deploy this project on a local machine, follow these steps:

```
## Setup and Run

1. **Create and activate virtual environment:**

```powershell
python -m venv venv
.\venv\Scripts\activate
```

2. Install dependencies:

```powershell
pip install flask
```

3. Run the application:

powershell
python app.py

4. Open in your browser:
http://127.0.0.1:5000/
```

16. Testing Strategy

- Manual testing of all pages and forms
- Chatbot queries tested for both rule-based and AI responses
- Admin dashboard functionality verified
- Database persistence checked for all interactions

17. Deployment Considerations

- Deploy using production WSGI server (e.g., Gunicorn)
- Use HTTPS with SSL certificates
- Environment variables for API and secret keys
- Consider scaling database if traffic increases

18. Limitations

- Passwords stored in plain text in current implementation
- SQLite is not suitable for very high traffic production systems
- AI responses depend on Google Gemini API availability

19. Conclusion

TechNova Solutions demonstrates a professional corporate website integrated with AI support, dynamic content management, and secure user authentication. It fulfills modern web application requirements and provides a scalable base for future development.

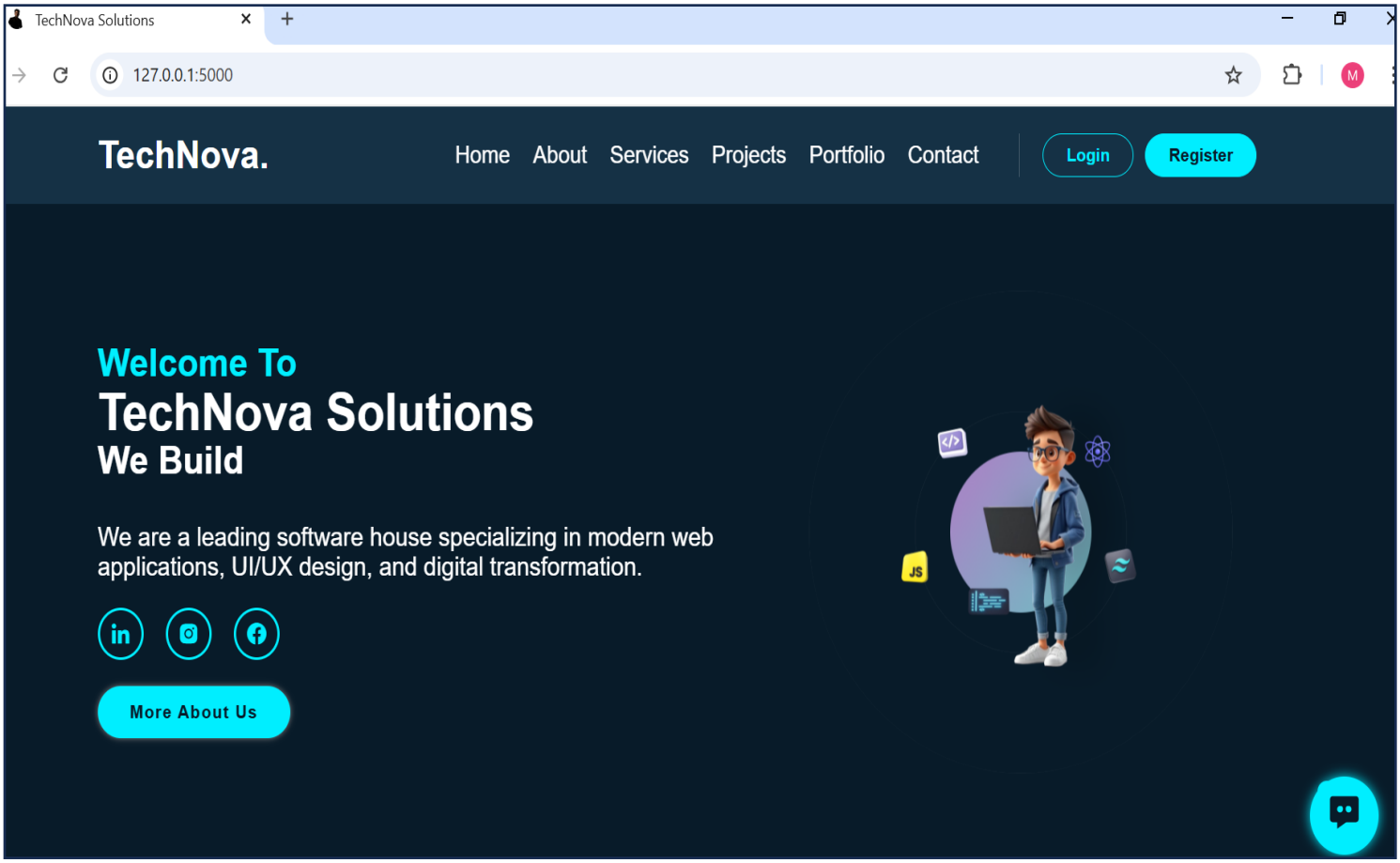
20. Future Scope

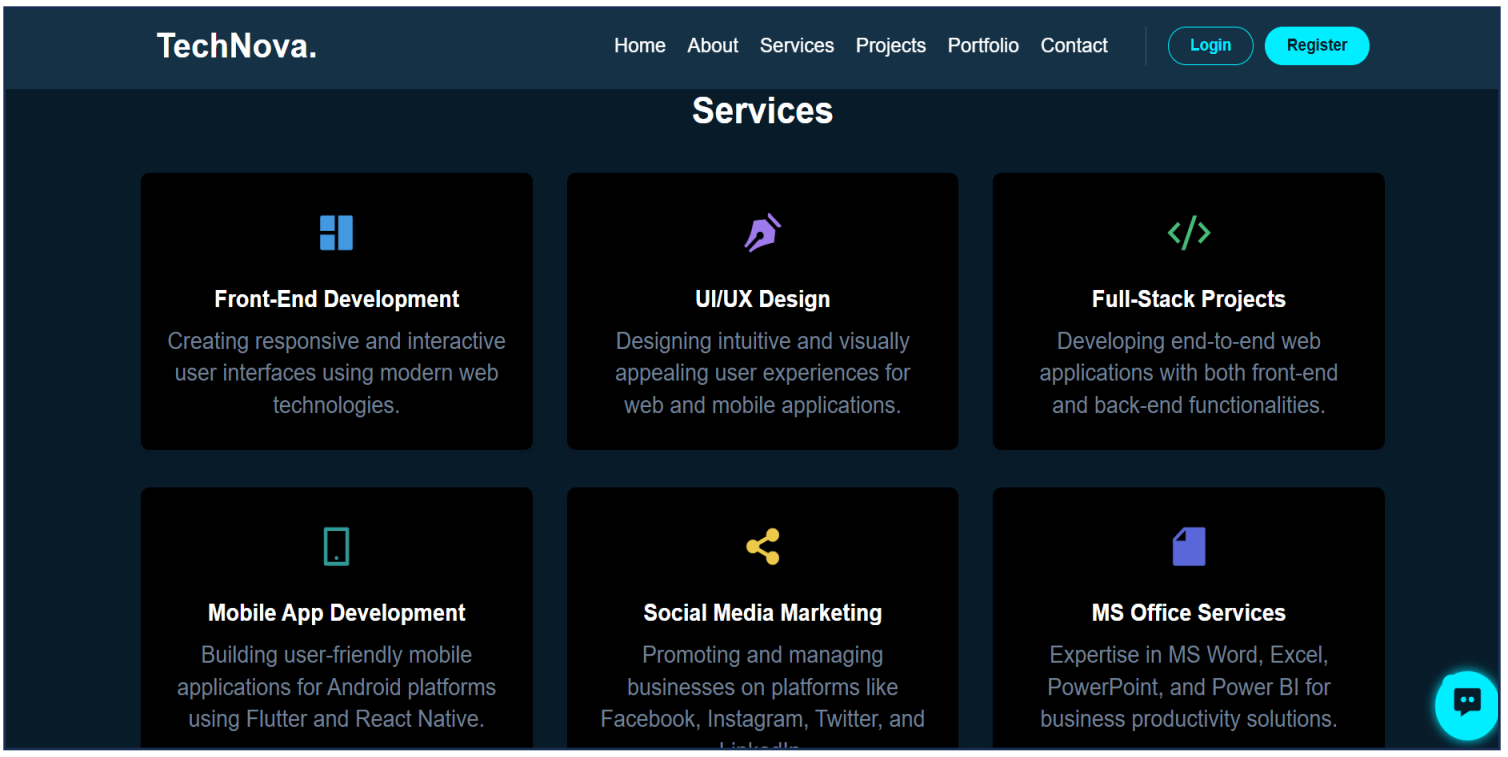
- Implement password hashing using werkzeug.security
- Move secret keys and API keys to environment variables
- Upgrade database to PostgreSQL or MySQL
- Admin features for replying/deleting messages directly

21. Project Directory Structure

Code

```
/Technova-Solutions
│
├── app.py                # Main application entry point
├── portfolio.db          # SQLite database file
│
├── /static
│   ├── style.css         # Main stylesheet
│   ├── script.js         # Client-side logic
│   └── /assets           # Images (Logo, Project screenshots)
│
└── /templates
    ├── base.html         # Base layout
    ├── index.html        # Homepage
    ├── login.html        # User Login Page
    ├── register.html     # User Registration Page
    ├── portfolio.html    # Team Portfolio Page
    └── admin_messages.html # Admin Dashboard
```


TechNova Solutions
127.0.0.1:5000
TechNova.
Home
About
Services
Projects
Portfolio
Contact
Login
Register
Welcome To
TechNova Solutions
We Build
We are a leading software house specializing in modern web applications, UI/UX design, and digital transformation.
More About Us


TechNova.
Home
About
Services
Projects
Portfolio
Contact
Login
Register
Services
Front-End Development
Creating responsive and interactive user interfaces using modern web technologies.
UI/UX Design
Designing intuitive and visually appealing user experiences for web and mobile applications.
Full-Stack Projects
Developing end-to-end web applications with both front-end and back-end functionalities.
Mobile App Development
Building user-friendly mobile applications for Android platforms using Flutter and React Native.
Social Media Marketing
Promoting and managing businesses on platforms like Facebook, Instagram, Twitter, and
MS Office Services
Expertise in MS Word, Excel, PowerPoint, and Power BI for business productivity solutions.